

Calculadora Científica Graficadora con ANTLR

Lenguajes de Programación y Traducción

Integrantes: Ángel Arcos – Yeimy Beltrán – Nicolás Gutiérrez – Samuel Lagos

1. Objetivo:

El objetivo del laboratorio es construir progresivamente un pequeño lenguaje matemático utilizando ANTLR. El proceso parte de una calculadora aritmética básica y avanza hasta incorporar números reales, variables, potencias, funciones científicas, constantes, comandos de memoria y representación gráfica.

2. Recordando la calculadora del libro

¿Por qué resulta conveniente tener un método diferente para una suma, una multiplicación y un número?

R//. Es conveniente porque cada expresión tiene un método diferente y necesita una acción distinta. Por ejemplo, no es lo mismo que el usuario ingrese un número, cuyo proceso es obtener su valor, a que ingrese una suma o una resta, donde se deben identificar los números y realizar la operación correspondiente. Esto permite que el Visitor esté organizado y que cada operación tenga su propia lógica, encargándose cada método de un tipo específico de expresión.

3. Creación de nuestro proyecto

Creamos la carpeta ScientificCalculator y los archivos necesarios

```
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos$  
mkdir ScientificCalculator  
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos$  
cd ScientificCalculator  
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/  
ScientificCalculator$ touch ScientificCal.g4  
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/  
ScientificCalculator$ touch main.java  
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/  
ScientificCalculator$ touch ScientificEvalVisitor.java  
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/  
ScientificCalculator$ touch PlotWindow.java  
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/  
ScientificCalculator$ touch ejemplos.txt  
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/  
ScientificCalculator$ ls  
ejemplos.txt  PlotWindow.java  ScientificEvalVisitor.java  
main.java    ScientificCal.g4  
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/  
ScientificCalculator$
```

3.1. Nuestra primera gramática

Determine si los siguientes textos pueden ser reconocidos por la regla ID:

Texto	¿Reconoce?	Justificación
variable	Sí	Porque comienza con una letra y sigue con caracteres permitidos ya que la regla dice que después de una letra puede contener más letras o números.
x2	Sí	Porque comienza con una letra del abecedario y la regla del ID dice que después puede contener números
2x	No	No reconoce el texto como ID, porque empieza con un número y la regla dice que el primer carácter debe ser una letra o un _ pero no un número.
_resultado	Sí	Porque la regla permite que el primer carácter sea letra o _ y después puede contener más letras, números o _.
variable-final	No	No acepta el texto completo porque aunque la regla se cumple hasta cierta parte contiene -, y ese carácter no está permitido dentro de ID.

3.2 Creación del visitor

Al ejecutar ANTLR con la opción -visitor se generan los archivos necesarios para recorrer el árbol sintáctico.

```
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/ScientificCalculator$ nano ScientificCalc.g4
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/ScientificCalculator$ antlr4 -no-listener -visitor ScientificCalc.g4
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/ScientificCalculator$ ls
ejemplos.txt          ScientificCalcLexer.java
main.java             ScientificCalcLexer.tokens
PlotWindow.java       ScientificCalcParser.java
ScientificCalcBaseVisitor.java ScientificCalc.tokens
ScientificCalc.g4     ScientificCalcVisitor.java
ScientificCalc.interp ScientificCal.g4
ScientificCalcLexer.interp
```

- Compare esos métodos con las etiquetas utilizadas en la gramática. ¿Qué relación encuentra?

R// Comparando los métodos que se generaron con las etiquetas que colocamos en el archivo .g4, podemos ver la relación directa entre ellos. Por ejemplo, la etiqueta que le colocamos a la suma, #addSub, genera el método visitAddSub(), mientras que la etiqueta #number genera visitNumber(), y así sucesivamente con cada una de las etiquetas.

```

T visitNumber(ScientificCalcParser.NumberContext c
/**
 * Visit a parse tree produced by the {@code paren
 * labeled alternative in {@link ScientificCalcPar
 * @param ctx the parse tree
 * @return the visitor result
 */
T visitParens(ScientificCalcParser.ParensContext c
/**
 * Visit a parse tree produced by the {@code addSu
 * labeled alternative in {@link ScientificCalcPar
 * @param ctx the parse tree
 * @return the visitor result
 */
T visitAddSub(ScientificCalcParser.AddSubContext c
/**
 * Visit a parse tree produced by the {@code id}
 * labeled alternative in {@link ScientificCalcPar
 * @param ctx the parse tree
 * @return the visitor result
 */
T visitId(ScientificCalcParser.IdContext ctx);
/**
 * Visit a parse tree produced by the {@code mulDi
 * labeled alternative in {@link ScientificCalcPar
 * @param ctx the parse tree
 * @return the visitor result

```

```

stat
: expr NEWLINE # printExpr
| ID '=' expr NEWLINE # assign
| NEWLINE # blank
;

expr
: expr op=('*' | '/') expr # mulDiv
| expr op=('+' | '-') expr # addSub
| NUMBER # number
| ID # id
| '(' expr ')' # parens
;

```

3.3 Implementación de multiplicación y división

Utilizando la misma estrategia, implemente:

- multiplicación
- división.

El método generado por ANTLR se denomina: visitMulDiv(...)

R// Se realizó la implementación de las operaciones de multiplicación y división dentro del método visitMulDiv(). Primero, obtenemos los valores tanto del primer valor (left) como del segundo (right) de la operación. Por medio de un if, verificamos si el operador corresponde a la multiplicación; si es así, se multiplican ambos valores. En caso contrario, se realiza la división, donde también colocamos un if para verificar que el segundo número ingresado sea diferente de 0, de manera que la división pueda realizarse. De lo contrario, no se realizará la división y se mostrará un mensaje de error.

```

@Override
public Double visitMulDiv(
    ScientificCalcParser.MulDivContext ctx) {

    double left = visit(ctx.expr(0));
    double right = visit(ctx.expr(1));

    if (ctx.op.getType() == ScientificCalcParser.MUL) {
        return left * right;
    }

    if (right == 0){
        throw new RuntimeException("No se puede dividir");
    }

    return left / right;
}

```

3.4 Prueba del primer intérprete

Se realizaron diferentes pruebas con expresiones básicas y con expresiones que permiten verificar la precedencia de los operadores. Por ejemplo, se probaron operaciones como $2+3$, $2+3*4$ y $(2+3)*4$. Los resultados obtenidos fueron los esperados de acuerdo con las reglas de la gramática, comprobando que el intérprete reconoce correctamente las expresiones.

```
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
2+2
10-3
10*5
20/4
2+3*4
(2+3)*4
4.0
7.0
50.0
5.0
14.0
20.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$
```

- Se realizaron pruebas adicionales. Una de ellas fue $8*0$, donde el intérprete reconoció correctamente la operación y el resultado fue 0.0.

```
ScientificCalculator$ java Main
8*0
0.0
```

También se realizó la prueba $10/0$, como anteriormente le habíamos agregado una condición para detectar la división entre cero y mostrar un mensaje de error, el intérprete mostró el mensaje “No se puede dividir entre cero”. De esta manera, comprobamos que la validación que habíamos agregado está funcionando correctamente.

```
ScientificCalculator$ java Main
10/0
Exception in thread "main" java.lang.RuntimeException: No se pu
ede dividir entre cero
    at ScientificEvalVisitor.visitMulDiv(ScientificEvalVisi
tor.java:145)
```

Por último, colocamos $(2+3$ para ver cómo es el comportamiento del intérprete ante un error de sintaxis. El intérprete sí indicó que esperaba el cierre del paréntesis, pero, sin embargo, también resolvió la suma y mostró como resultado 5.0.

```
ScientificCalculator$ java Main
(2+3(
line 1:4 mismatched input '(' expecting {'}', '*', '/', '+', '-'
'}
5.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
```

3.5 Comprobación de implementación de variables

Se realizó la verificación del manejo de variables, comprobando que el intérprete reconoce las variables correctamente y muestra su valor cuando han sido definidas. Los resultados obtenidos fueron los esperados.

```
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ javac *.java
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
a=10
b=20
a+b
a*b
30.0
200.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$
```

- ¿Qué sucedería si escribiera: **resultado + 10** sin haber asignado previamente un valor a resultado?

¿Considera adecuado devolver cero o sería mejor producir un error?

R// Si escribiera resultado + 10 sin haber asignado previamente un valor a resultado, el programa mostraría un mensaje de variable no definida y devuelve 0 y como el 10 es un número que el programa puede reconocer, al hacer la suma daría como resultado 10.

Pero desde el punto de vista de diseño sería más claro producir un error explícito, porque una variable inexistente y una variable cuyo valor real es cero no representan la misma situación.

```
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/ScientificCalc
java Main
resultado+10
Variable no definida: resultado
10.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$
```

3.6 Comprobación de potencia

Determine manualmente el resultado esperado para cada expresión antes de ejecutarla.
Compare posteriormente su respuesta con la calculadora.

Comprobación manual

- $2^8 = 2*2*2*2*2*2*2*2 = 256$
- $10^2 = 10*10 = 100$
- $2^3 + 4 = 2*2*2 + 4 = 12$
- $2*3^2 = 2*9 = 18$

Comprobación con la calculadora

```
ScientificCalculator$ javac *.java
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
2^8
10^2
2^3+4
2*3^2
256.0
100.0
12.0
18.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$
```

R// Se realizaron las operaciones de manera manual y luego se ejecutaron en la calculadora. Al comparar los resultados obtenidos, se comprobó que coinciden en todos los casos, con esto podemos verificar que, después de incluir la operación de potencia, el programa está dando resultados correctos.

3.7 Comprobación de Funciones

Se realizaron las pruebas de las funciones `sqrt`, `cos`, `log` y `abs` en la calculadora y para casi todas las funciones producen correctamente los resultados esperados, excepto la función de valor absoluto, ya que se produce un error porque aún no se ha implementado para recibir valores negativos. Sin embargo, el intérprete continúa la ejecución y obtiene como resultado 10.

```

yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
sqrt(25)
cos(0)
log(100)
abs(-10)
line 4:4 extraneous input '-' expecting {'(', 'sin', 'cos', 'ta
n', 'sqrt', 'log', 'ln', 'abs', 'exp', NUMBER, ID}
5.0
1.0
2.0
10.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ 

```

3.8 Comprobación luego de implementar número negativos

Se realizaron las comprobaciones de la inclusión de los números negativos y se verificó que la implementación quedó correcta. Para comprobarlo, se realizaron varias pruebas utilizando números negativos en diferentes operaciones, como sumas, restas y multiplicaciones, y en todos los casos se obtuvieron los resultados esperados por lo que podemos comprobar que la calculadora ahora reconoce correctamente los números negativos

```

yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
-10
abs(-10)
-2+5
-5*2
-10.0
10.0
3.0
-10.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ 

```

3.9 Comprobación de nuestra primera calculadora científica

Se realizó la implementación de las diferentes constantes matemáticas pi y e, las funciones matemáticas, la raíz cuadrada y la potencia. Para comprobar que estas implementaciones funcionaran correctamente se realizaron las pruebas indicadas en el tutorial y se obtuvieron los resultados esperados. Con esto se pudo comprobar que la calculadora ya puede reconocer y realizar este tipo de operaciones correctamente.


```

yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
pi
2*pi
sin(pi/2)
cos(0)
log(100)
ln(e)
sqrt(25)
2^8

3.141592653589793
6.283185307179586
1.0
1.0
2.0
1.0
5.0
256.0

```

3.10 Comprobación del comando clear

Se realizó la implementación de la función clear. Aunque la función está funcionando de manera correcta, se está presentando un pequeño error relacionado con el salto de línea, ya que el programa está identificando que falta un NEWLINE antes de la siguiente instrucción. Sin embargo, este error no afecta el funcionamiento de la función, ya que después de ejecutar clear se borra correctamente la memoria. Esto se puede comprobar porque al volver a utilizar las variables a y b, el programa indica que las variables no están definidas. De esta manera, se pudo comprobar que la implementación de clear se realizó correctamente y que cumple con la función de eliminar las variables almacenadas.

```

yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
a=10
b=20
a+b

clear
a+b

line 6:0 missing NEWLINE at 'a'
30.0
Memoria eliminada.
0.0
Variable no definida: a
Variable no definida: b
0.0

```


3.11 Comprobación del comando vars

Se realizó la implementación del comando vars, el cual permite mostrar las variables que se encuentran almacenadas en la memoria del programa. Se realizó la comprobación y como resultado, el programa mostró correctamente las tres variables almacenadas junto con sus valores. También se presenta un mensaje relacionado con el salto de línea al finalizar la entrada, pero este no impide que el comando vars funcione correctamente, ya que las variables son mostradas de manera adecuada.

```
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/ScientificCalculator$ java Main
a=10
b=20
radio=5

vars
line 6:0 missing NEWLINE at '<EOF>'
a = 10.0
b = 20.0
radio = 5.0
0.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/ScientificCalculator$
```

3.12 Modificación pequeña del código

Modifique el código de muestreo para que solamente almacene valores válidos.

Puede utilizar:

```
if (Double.isFinite(y)) {
    xs.add(x);
    ys.add(y);
}
```

Se realizó la implementación de un if para verificar que los valores obtenidos al evaluar la función sean válidos. Para esto se utilizó Double.isFinite(y), que permite comprobar que el resultado no sea un valor como Infinity, -Infinity o NaN. De esta manera, solamente se guardan en las listas los valores que sean válidos para posteriormente poder realizar la gráfica correctamente

```

        memory.put("x", x);

        double y =
            visit(ctx.expr(0));

        if (Double.isFinite(y)) {
            xs.add(x);
            ys.add(y);
        }
    }

    new PlotWindow(xs, ys);

    return 0.0;
}
}

```

- ¿Qué efecto tiene esta modificación sobre la gráfica de $1/x$?

R// La gráfica se divide en dos partes, una para $x < 0$ y otra para $x > 0$ por que se eliminan los valores no válidos, como por ejemplo el punto $x=0$ donde $1/x$ no está definido, esto evita que la gráfica pueda conectar incorrectamente las dos partes.

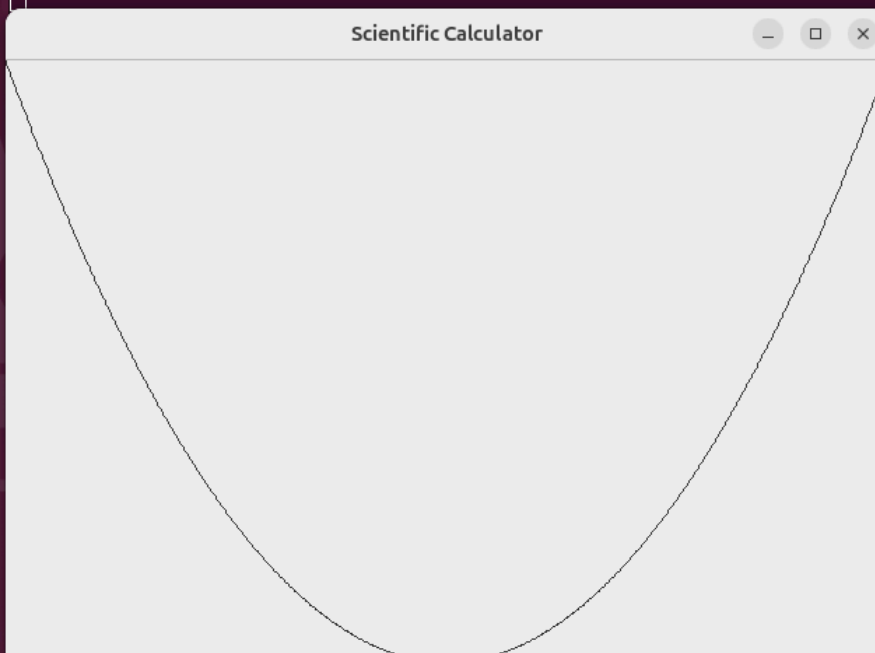
3.13 Primera gráfica

Realizamos la prueba de nuestra primera gráfica después de todas las implementaciones y comprobamos que funciona correctamente, ya que la función x^2 corresponde a una parábola como la que observamos en la gráfica. Con esto verificamos que la función se está graficando correctamente.

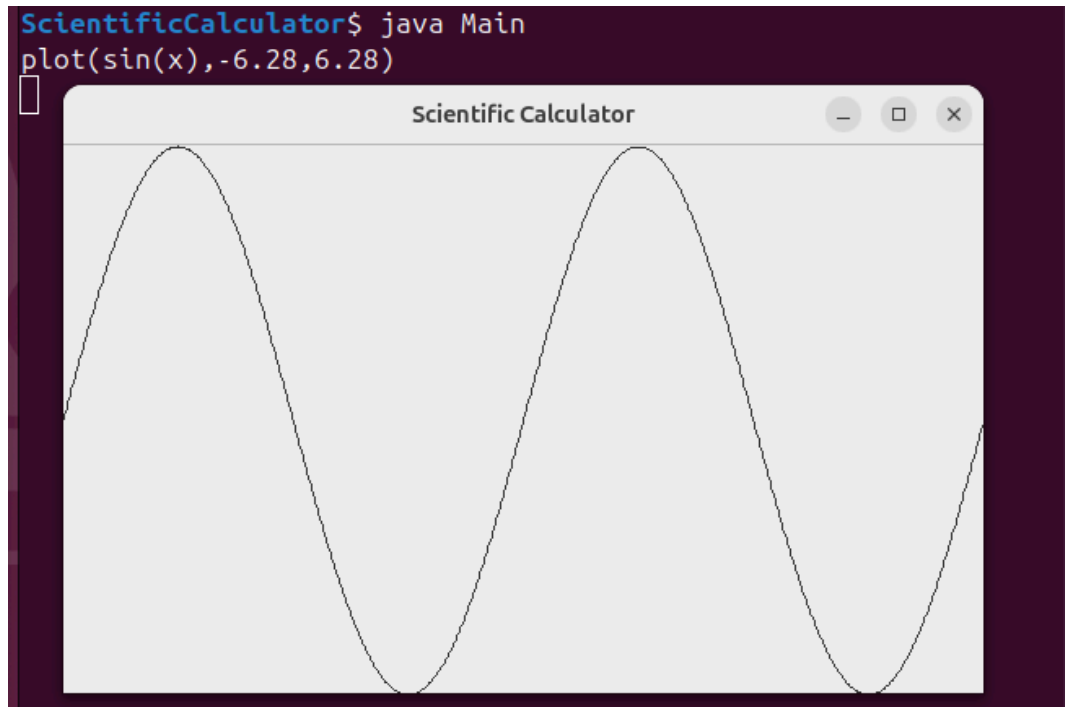
```

yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ javac *.java
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
plot(x^2,-10,10)

```



Posteriormente, probamos la función seno, la cual corresponde con la gráfica que obtuvimos. Por lo tanto, comprobamos que también está funcionando correctamente.



3.14 Cuando se ejecuta el Visitor:

`visit(ctx.expr())`

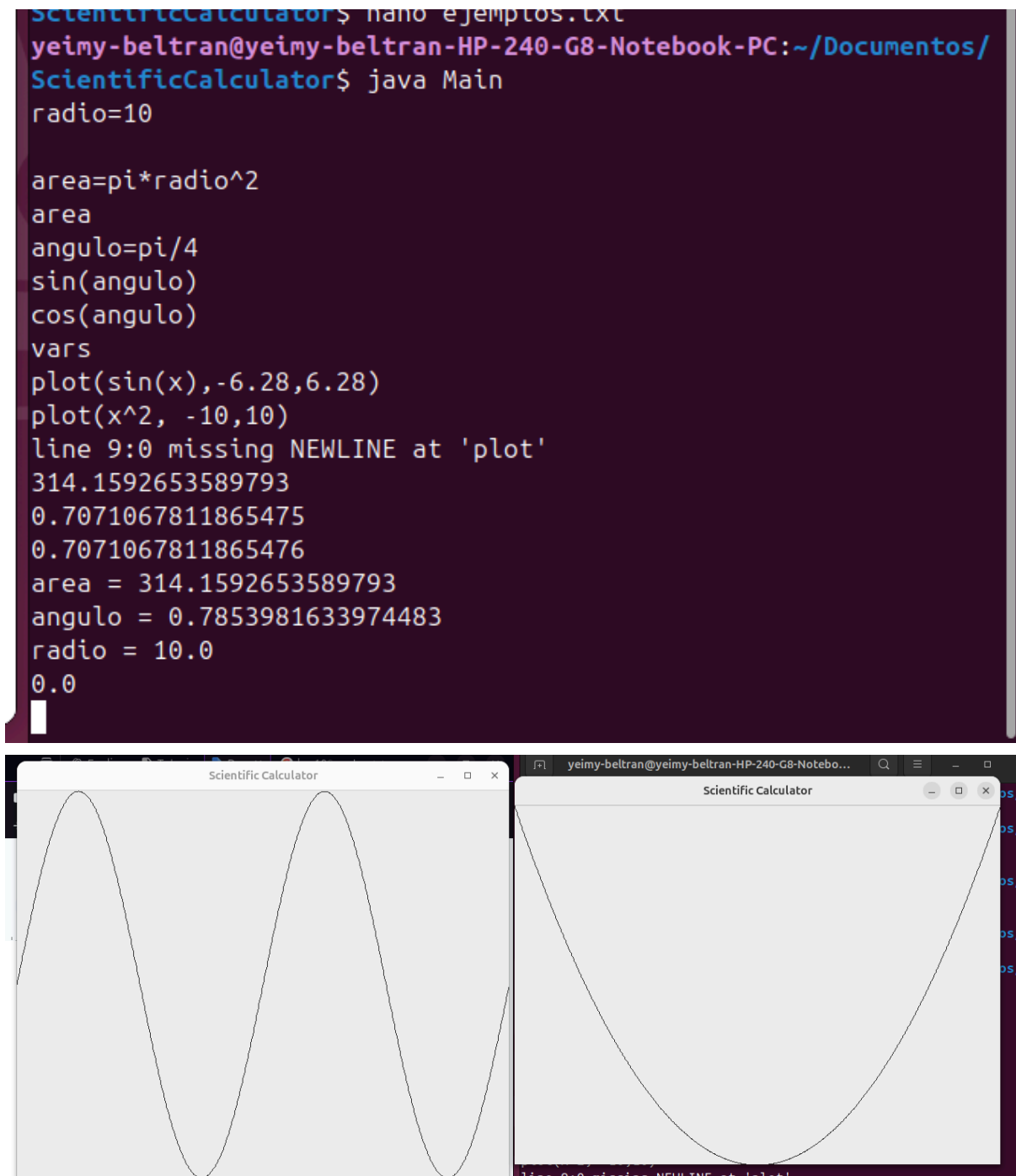
¿Se evalúa una cadena de texto o se visita una estructura de árbol?, Explique su respuesta

R// Se visita una estructura de árbol, no una cadena de texto. Primero, el Lexer identifica las entradas y las convierte en tokens, y el Parser verifica que estos tokens cumplan con la gramática y construye el árbol sintáctico. Luego, el Visitor recorre ese árbol mediante `visit(ctx.expr())` para evaluar la expresión y realizar la operación correspondiente.

3.15 Comprobación final de todo el lenguaje

Se realizó la comprobación final del funcionamiento de la calculadora utilizando diferentes instrucciones del lenguaje. Primero se verificó la asignación de variables, posteriormente se realizaron operaciones utilizando estas variables y finalmente se probaron funciones matemáticas como sin y cos. También se comprobó el funcionamiento de la gráfica con diferentes expresiones.

Como se puede observar en las pruebas, la calculadora logró procesar correctamente las diferentes instrucciones planteadas. Esto nos permite comprobar que las partes implementadas anteriormente funcionan en conjunto y que el programa no solamente realiza operaciones individuales, sino que también puede interpretar varias instrucciones de un pequeño lenguaje matemático.



3.16 . Preguntas finales

Responda las siguientes preguntas.

1. ¿Cuál es la responsabilidad del Lexer?

R// El Lexer se encarga de tomar el texto que se escribe por consola y dividirlo en tokens, identificando qué representa cada parte. Por ejemplo, en $2 + 3$, identifica 2 como un número, + como un operador y 3 como otro número.

2. ¿Cuál es la responsabilidad del Parser?

R// Verificar si los tokens recibidos por el Lexer cumplen con las reglas de la gramática que se establecieron y si son válidos para que se pueda construir el árbol sintáctico. Por ejemplo, si recibieron $2+3$, verifica que la expresión tenga una estructura válida.

3. ¿Qué función cumplen las etiquetas como #addSub o #functionCall?

R// Las etiquetas nos permiten identificar qué tipo de operación o expresión representa cada parte del árbol sintáctico. Además hacen que ANTLR genere métodos específicos en el Visitor para procesar cada paso. Por ejemplo, de la etiqueta #addSub se genera el visitAddSub() para manejar las sumas y restas, mientras que la etiqueta #functionCall se genera visitFunctionCall() para manejar las funciones como $\sin(x)$ o $\sqrt{x}$.

4. ¿Qué ventaja ofrece el patrón Visitor?

R// Permite recorrer y procesar el árbol sintáctico sin tener que colocar toda la lógica de las operaciones dentro de la gramática. Por ejemplo, podemos agregar una nueva función como $\sqrt{x}$ implementando su comportamiento en el visitor.

5. ¿Qué representa la tabla de símbolos?

R// Es la memoria donde se almacenan las variables definidas por el usuario junto con sus respectivos valores. Por ejemplo, si tenemos $\text{area}=10$, la tabla de símbolos almacena que area corresponde a 10 y cuando después hagamos una operación como $\text{area}*2$, el visitor consulta la tabla para obtener el valor de área y realizar la operación.

6. ¿Por qué la variable x cambia continuamente durante una gráfica?

R// La variable x cambia continuamente porque necesitamos evaluar la función para diferentes valores de x y así obtener los diferentes puntos que forman la gráfica. Por ejemplo, para graficar $y = x^2$, podemos evaluar la función con $x = -2$, luego $x = -1$, $x = 0$, $x = 1$, $x = 2$, etc. Cada valor de x produce un valor diferente de y.

7. ¿Por qué podemos evaluar el mismo árbol sintáctico varias veces?

R// Podemos evaluar el mismo árbol sintáctico varias veces porque la estructura de la expresión no cambia; lo que cambia son los valores de las variables que utiliza. Por ejemplo, para x^2 , podemos asignar primero $x = 2$ y obtener 4, después cambiar $x = 3$ y obtener 9, utilizando el mismo árbol sintáctico.

8. ¿Qué sucede cuando se intenta graficar una función con una discontinuidad?

R// Cuando se intenta graficar una función con una discontinuidad, pueden aparecer valores que no son finitos. Por eso se deben ignorar esos valores para evitar que la gráfica conecte puntos que realmente no están conectados. Por ejemplo, en $1/x$, la función no está definida cuando $x = 0$, por lo que la gráfica queda separada en dos partes: una para $x < 0$ y otra para $x > 0$.

9. ¿Qué modificaciones serían necesarias para implementar funciones con dos argumentos?

R// Sería necesario modificar la gramática para permitir que una función reciba dos expresiones separadas por una coma y después modificar el Visitor para evaluar ambos argumentos y realizar la operación correspondiente. Por ejemplo, para permitir $\text{pow}(2,8)$, la gramática tendría que reconocer los dos argumentos y el Visitor podría obtener el valor de cada uno para calcular la potencia.

10. ¿Por qué la calculadora desarrollada puede considerarse un lenguaje de dominio específico?
R// Porque está diseñada específicamente para trabajar con operaciones y expresiones matemáticas, como $\sin(x)$, $\sqrt{25}$ y x^2 , en lugar de ser un lenguaje para cualquier tipo de problema.

4. Reto 1 - Nuevas funciones

Agregue:

- a. $\text{asin}(x)$
- b. $\text{acos}(x)$
- c. $\text{atan}(x)$
- d. $\text{floor}(x)$
- e. $\text{ceil}(x)$

En este reto se agregaron cinco nuevas funciones matemáticas: asin , acos , atan , floor y ceil . Primero se agregaron a la regla función para que la calculadora pudiera reconocer estas nuevas funciones.

```
function
  : 'sin'
  | 'cos'
  | 'tan'
  | 'sqrt'
  | 'log'
  | 'ln'
  | 'abs'
  | 'exp'
  | 'asin'
  | 'acos'
  | 'atan'
  | 'floor'
  | 'ceil'
  ;
```

Adicionalmente se agregaron las nuevas funciones dentro del `switch` en el método `visitFunctionCall`, cuando el usuario escribe una de estas funciones, el `Visitor` identifica cuál es y realiza la operación correspondiente.

```

public Double visitFunctionCall(
    switch (function) {
        case "log":
            return Math.log(value);
        case "ln":
            return Math.log(value);
        case "abs":
            return Math.abs(value);
        case "exp":
            return Math.exp(value);
        case "asin":
            return Math.asin(value);
        case "acos":
            return Math.acos(value);
        case "atan":
            return Math.atan(value);
        case "floor":
            return Math.floor(value);
        case "ceil":
            return Math.ceil(value);
        default:
            throw new RuntimeException(

```

Se realizaron varias pruebas para comprobar el funcionamiento de las cinco funciones agregadas. Se utilizaron diferentes valores como entrada y se obtuvieron los resultados correspondientes para cada operación. Por ejemplo, $\text{asin}(1)$ obtuvo aproximadamente 1.5708, $\text{acos}(1)$ obtuvo 0.0, $\text{atan}(1)$ obtuvo aproximadamente 0.7854, $\text{floor}(3.8)$ obtuvo 3.0 y $\text{ceil}(3.2)$ obtuvo 4.0. Con estos resultados se comprobó que las nuevas funciones fueron reconocidas y ejecutadas correctamente por la calculadora.

```

yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ javac *.java
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
asin(1)
acos(1)
atan(1)
floor(3.8)
ceil(3.2)
1.5707963267948966
0.0
0.7853981633974483
3.0
4.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$

```


5. Reto 2 – Funciones con dos argumentos

Diseñe una extensión para:

- `pow(2,8)`
- `max(10,25)`
- `min(10,25)`

Antes de programar, escriba primero la regla de la gramática que necesitaría

R// Primero se planteó la modificación de la regla de gramática `functionCall`. La regla de gramática que se utilizaría para este caso sería `function '(' expr ',' expr ')' #functionCall`, ya que, en comparación con la regla original que tenían las funciones, `function '(' expr ')' #functionCall`, esta solo acepta un argumento y no los dos necesarios para este reto.

Luego del planteamiento, se realizó el cambio en el Visitor para poder trabajar con los dos valores que recibe la función. Para esto, se tomaron las dos expresiones y se obtuvo el valor de cada una. Después, en el switch, se agregaron las funciones `pow`, `max` y `min`, indicando qué debía hacer la calculadora con los dos valores. De esta manera, la calculadora puede trabajar con funciones que necesitan dos argumentos.

```
@Override
public Double visitFunctionCall(
    ScientificCalcParser.FunctionCallContext ctx) {
    String function = ctx.function().getText();

    double value1 = visit(ctx.expr(0));

    if (ctx.expr().size() == 2) {
        double value2 = visit(ctx.expr(1));

        switch (function) {
            case "pow":
                return Math.pow(value1, value2);
            case "max":
                return Math.max(value1, value2);
            case "min":
                return Math.min(value1, value2);
            default:
                throw new RuntimeException(
                    "Funcion desconocida: " + function);
        }
    }
}
```

```
function
: 'sin'
| 'cos'
| 'tan'
| 'sqrt'
| 'log'
| 'ln'
| 'abs'
| 'exp'
| 'asin'
| 'acos'
| 'atan'
| 'floor'
| 'ceil'
| 'pow'
| 'max'
| 'min'
;
```

Finalmente, se realizaron pruebas con las funciones pow, max y min, utilizando dos valores en cada una. Los resultados obtenidos fueron los esperados. Por ejemplo, se probó pow(2,8) y se obtuvo 256, mientras que max y min devolvieron correctamente el valor mayor y menor entre los dos números ingresados.

```
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/ScientificCalculator$ java Main
pow(2,8)
max(10,25)
min(10,25)
256.0
25.0
10.0
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/ScientificCalculator$
```

6. Reto 3 – Rango vertical

Amplíe:

plot(expr,xmin,xmax)

a:

plot(expr,xmin,xmax,ymin,ymax)

R// En este reto se modificó el comando plot para permitir que el usuario también pueda establecer el rango vertical de la gráfica mediante ymin y ymax. De esta manera, además de indicar el rango de x, se puede controlar qué parte de los valores de y se muestra en la gráfica.

```
yeimy-beltran > Documentos > ScientificCalculator > PlotWindow.java
import javax.swing.*;
import java.awt.*;
import java.util.List;

public class PlotWindow extends JPanel {

    private List<Double> xs;
    private List<Double> ys;

    private double ymin;
    private double ymax;

    public PlotWindow(
        List<Double> xs,
        List<Double> ys,
        double ymin,
        double ymax) {

        this.xs = xs;
        this.ys = ys;
        this.ymin = ymin;
        this.ymax = ymax;

        JFrame frame =
```

En la prueba se utilizó la función x^2 con un rango horizontal de -10 a 10 y un rango vertical de 0 a 100. Al establecer estos valores, se pudo observar que la gráfica se ajusta a los límites indicados, mostrando únicamente la parte de la función que se encuentra dentro del rango vertical establecido. Esto permitió comprobar que la modificación realizada al comando plot funciona correctamente y que ahora el usuario puede controlar tanto el rango de x como el de y al momento de generar una gráfica.



7. Reto 4 – Varias funciones

Se diseñó una sintaxis para poder escribir:

```
plot(sin(x),cos(x),-6.28,6.28)
```

No modifique primero el código Java.

Se comenzó diseñando la gramática.

En este reto se planteó una modificación en la gramática para permitir que el comando plot pueda recibir dos funciones al mismo tiempo. La idea es poder escribir una expresión como `plot(sin(x),cos(x),-6.28,6.28)`, donde se tienen dos funciones y el rango en el que se desea realizar la gráfica. Primero se diseñó la regla de gramática necesaria para reconocer estos elementos y posteriormente se realizó la implementación en el código Java.

```

stat
: expr NEWLINE # printExpr
| ID '=' expr NEWLINE # assign
| 'plot' '(' expr ',' expr ',' expr ',' expr ',' expr ')' NEWLINE # plotExpr
| NEWLINE # blank
| 'plot' '(' expr ',' expr ',' expr ',' expr ')' NEWLINE # plotTwoExpr
;

```

```

@Override
public Double visitPlotTwoExpr(
    ScientificCalcParser.PlotTwoExprContext ctx) {

    double xmin = visit(ctx.expr(2));
    double xmax = visit(ctx.expr(3));

    int samples = 800;

    List<Double> xs = new ArrayList<>();
    List<Double> ys1 = new ArrayList<>();
    List<Double> ys2 = new ArrayList<>();

    for (int i = 0; i < samples; i++) {

        double x =
            xmin
            + i * (xmax - xmin)
            / (samples - 1);

        memory.put("x", x);

        double y1 = visit(ctx.expr(0));
        double y2 = visit(ctx.expr(1));

        xs.add(x);
        ys1.add(y1);
        ys2.add(y2);
    }
}

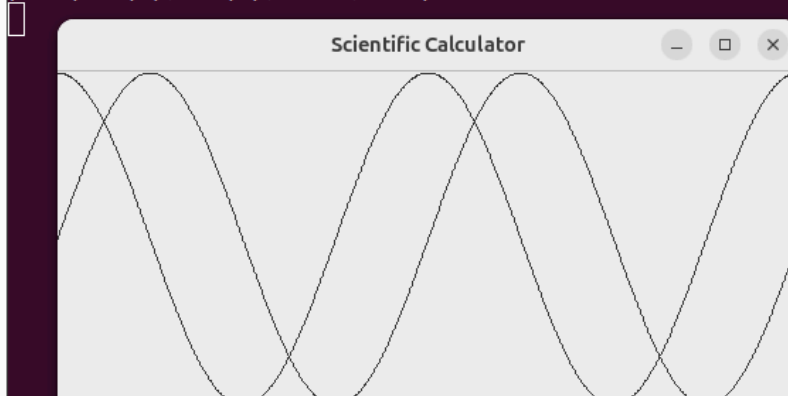
```

Como resultado, se logró que la calculadora pudiera graficar dos funciones al mismo tiempo utilizando un solo comando plot. En la prueba realizada se pueden observar las gráficas de $\sin(x)$ y $\cos(x)$ dentro del rango establecido de -6.28 a 6.28. Esto permite comprobar que la calculadora reconoce correctamente las dos funciones, las procesa y las representa en una misma gráfica.

```

yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ javac *.java
yeimy-beltran@yeimy-beltran-HP-240-G8-Notebook-PC:~/Documentos/
ScientificCalculator$ java Main
plot(sin(x),cos(x),-6.28,6.28)

```



8. Reto 5 – Definir funciones

¿Cómo podría evolucionar nuestro lenguaje para permitir?

$f(x) = x^2 + 2x + 1$

$f(5)$

$\text{plot}(f(x), -10, 10)$

Se realizó inicialmente solamente el diseño sintáctico.

R// Primero se diseñó la sintaxis necesaria para permitir que el lenguaje pueda definir funciones creadas por el usuario y posteriormente utilizarlas. Para esto, se agregaron dos nuevas reglas primera `functionDef`, que permite definir una función indicando su nombre, su variable y la expresión que la compone, y `functionCallUser`, que permite llamar una función utilizando un valor como argumento. Con estas reglas se busca poder reconocer expresiones como $f(x) = x^2 + 2x + 1$ y posteriormente realizar una llamada como $f(5)$.

```
'pow'
'max'
'min'
;

functionDef
: ID '(' ID ')' '=' expr
;

functionCallUser
: ID '(' expr ')'
;
```

La regla `functionDef` permite escribir la definición de una función, por ejemplo $f(x) = x^2 + 2x + 1$, donde f es el nombre de la función, x es la variable y la expresión corresponde a la operación que realizará. Por otro lado, `functionCallUser` permite utilizar una función ya definida, como en $f(5)$, pasando un valor para la variable.

9. Conclusión

El desarrollo del tutorial permitió pasar de una calculadora aritmética sencilla a un pequeño lenguaje matemático construido alrededor de ANTLR y el patrón Visitor. La evolución fue incremental: primero se reconocieron expresiones básicas; después se incorporaron variables, potencia, funciones, operadores unarios, constantes y comandos; finalmente se añadió la capacidad de muestrear y graficar funciones.

Uno de los aprendizajes principales es la relación entre gramática y árbol sintáctico. La gramática determina qué expresiones son válidas y las etiquetas permiten identificar sus diferentes estructuras. El Parser convierte esa estructura en un árbol y el Visitor se encarga de darle significado mediante la evaluación de cada tipo de nodo.